



UNIVERSIDAD SIMÓN BOLÍVAR
DECANATO DE ESTUDIOS PROFESIONALES
COORDINACIÓN DE INGENIERÍA DE COMPUTACIÓN

**MOTOR DE TRIAJE DE ALERTAS DE SEGURIDAD CON MODELOS DE
LENGUAJE Y RECUPERACIÓN AUMENTADA LOCAL**

Por:

Blanyer Vielma

INFORME DE PASANTÍA

Presentado ante la Ilustre Universidad Simón Bolívar
como requisito parcial para optar al título de

Ingeniero de Computación

Sartenejas, Septiembre de 2026



UNIVERSIDAD SIMÓN BOLÍVAR
DECANATO DE ESTUDIOS PROFESIONALES
COORDINACIÓN DE INGENIERÍA DE COMPUTACIÓN

**MOTOR DE TRIAJE DE ALERTAS DE SEGURIDAD CON MODELOS DE
LENGUAJE Y RECUPERACIÓN AUMENTADA LOCAL**

Por:

Blanyer Vielma

Realizado con la asesoría de:

Tutor Académico: Por confirmar con la Coordinación

Tutor Industrial: Por confirmar con la empresa

INFORME DE PASANTÍA

Presentado ante la Ilustre Universidad Simón Bolívar

como requisito parcial para optar al título de

Ingeniero de Computación

Sartenejas, Septiembre de 2026

*[Espacio reservado para el Acta de Evaluación de Pasantía,
con las firmas originales del jurado.]*

Resumen

Las organizaciones enfrentan un volumen de alertas de seguridad que supera la capacidad de análisis manual de sus equipos, en buena parte porque los sistemas basados en reglas y firmas generan un alto volumen de falsos positivos sin justificar sus decisiones. Este trabajo, realizado como pasantía larga en Domotai C.A., desarrolla un motor de triaje que clasifica y prioriza alertas de seguridad, justifica cada decisión con un modelo de lenguaje de gran escala (LLM) apoyado en generación aumentada por recuperación (RAG) ejecutada en local, y propone una acción de respuesta contenida sujeta a validación humana: el modelo justifica, no decide. La metodología partió de modelar un cliente genérico ante la ausencia de un módulo propietario disponible, revisó el estado del arte de la detección extendida y gestionada y de los modelos de lenguaje en seguridad, construyó un laboratorio reproducible con Containerlab y Wazuh como fuente de alertas, diseñó una arquitectura que separa clasificación, política de decisión determinista y perfil de cliente configurable, e implementó y evaluó el prototipo frente al nivel de regla de Wazuh. Sobre 205 alertas reales, el prototipo redujo la tasa de falsos positivos de 0,282 a 0,041 —casi siete veces menos— sin dejar de detectar ninguna amenaza y sin acciones disruptivas indebidas. La recuperación aumentada corrigió el defecto principal del justificador: el modelo ancla sus explicaciones en la alerta pero, sin ella, las describe con imprecisión. Queda como limitación el clasificador con ajuste fino, bloqueado por un conjunto de datos con una sola familia de ataque.

Palabras clave: triaje de alertas de seguridad, modelos de lenguaje de gran escala, generación aumentada por recuperación, MITRE ATT&CK, validación humana.

ÍNDICE GENERAL

Resumen	IV
Índice de tablas	VII
Lista de símbolos	VIII
Lista de abreviaturas	IX
Introducción	1
1. Descripción de la empresa	6
1.1. Identificación y ubicación	6
1.2. Actividad y servicios	6
1.3. Modelo de trabajo	7
1.4. Ámbito de la pasantía	7
2. Marco teórico	9
2.1. El problema del triaje de alertas de seguridad	9
2.2. Detección y respuesta: EDR, XDR y MDR	10
2.3. MITRE ATT&CK como taxonomía de referencia	10
2.4. Modelos de lenguaje de gran escala aplicados a la seguridad	11
2.5. Generación aumentada por recuperación	12
2.6. Riesgos de aplicar un modelo de lenguaje al triaje de seguridad	13
2.7. Reglas y firmas frente a análisis asistido por modelos de lenguaje	14
2.8. Políticas de decisión determinista y separación entre analizar y actuar	14
2.9. Métricas de evaluación de un clasificador binario	15

2.10. Entornos de red virtualizados para experimentación en seguridad	16
3. Metodología	17
3.1. Enfoque general: modelar en lugar de esperar	17
3.2. Fase 1: modelado del cliente y del caso de uso	17
3.3. Fase 2: estado del arte y registro de requisitos	18
3.4. Fase 3: construcción del entorno de pruebas	18
3.5. Fase 4: diseño de la arquitectura	19
3.6. Fase 5: implementación	20
3.7. Fase 6: evaluación	22
4. Resultados y discusión	23
4.1. Resultado principal: clasificación frente al método de referencia	23
4.2. El límite de precisión del prototipo, y por qué	24
4.3. Por qué el óptimo del baseline es débil	24
4.4. Operación y continuidad del servicio	24
4.5. Calidad de la justificación: anclada, pero inicialmente poco fiable	25
4.6. El contraste con recuperación aumentada	26
4.7. Verificación sobre el conjunto completo	26
4.8. Limitaciones y análisis de errores	27
Conclusiones y recomendaciones	28
Referencias	30

ÍNDICE DE TABLAS

1.1. Líneas de servicio de Domotai C.A.	7
2.1. Reglas y firmas frente a análisis asistido por modelos de lenguaje	14
2.2. Métricas de clasificación empleadas en la evaluación	15
4.1. Prototipo frente al baseline de Wazuh (partición de evaluación, $n = 205$) . .	23
4.2. Medida F_1 del baseline según el umbral de nivel de regla	24

Lista de símbolos

Símbolos latinos

F_1	Media armónica de la precisión y la exhaustividad
FN	Número de falsos negativos de la matriz de confusión
FP	Número de falsos positivos de la matriz de confusión
k	Número de pasajes recuperados por consulta en el módulo de generación aumentada por recuperación
n	Número de alertas de una muestra o de una campaña de evaluación
P	Precisión de la clasificación
R	Exhaustividad (<i>recall</i>) de la clasificación
VN	Número de verdaderos negativos de la matriz de confusión
VP	Número de verdaderos positivos de la matriz de confusión

Lista de abreviaturas

Sigla	Significado
ACS	<i>Auto Configuration Server</i> — servidor de autoconfiguración remota de equipos de cliente bajo TR-069/TR-369
API	<i>Application Programming Interface</i> — interfaz de programación de aplicaciones
ATT&CK	<i>Adversarial Tactics, Techniques, and Common Knowledge</i> — matriz de tácticas y técnicas adversarias de MITRE
CLI	<i>Command-Line Interface</i> — interfaz de línea de comandos
CPU	<i>Central Processing Unit</i> — unidad central de procesamiento
CVE	<i>Common Vulnerabilities and Exposures</i> — identificador público de una vulnerabilidad conocida
DDoS	<i>Distributed Denial of Service</i> — denegación de servicio distribuida
DMZ	<i>Demilitarized Zone</i> — zona desmilitarizada de una red
DORA	<i>Digital Operational Resilience Act</i> — reglamento europeo de resiliencia operativa digital del sector financiero
EDR	<i>Endpoint Detection and Response</i> — detección y respuesta en el <i>endpoint</i>
FN	Falso negativo
FP	Falso positivo
GB	Gigabyte
GPU	<i>Graphics Processing Unit</i> — unidad de procesamiento gráfico
HIDS	<i>Host-based Intrusion Detection System</i> — sistema de detección de intrusiones basado en <i>host</i>
IoT	<i>Internet of Things</i> — internet de las cosas
IP	<i>Internet Protocol</i> — protocolo de internet
JSON	<i>JavaScript Object Notation</i> — notación de objetos de JavaScript
LLM	<i>Large Language Model</i> — modelo de lenguaje de gran escala
MDR	<i>Managed Detection and Response</i> — detección y respuesta gestionada

Sigla	Significado
NIS2	<i>Network and Information Security Directive 2</i> — directiva europea de seguridad de redes y sistemas de información
NIST	<i>National Institute of Standards and Technology</i> — instituto nacional de normas y tecnología de los Estados Unidos
ODF	<i>Optical Distribution Frame</i> — repartidor óptico; punto de entrega del proveedor de servicio
OT	<i>Operational Technology</i> — tecnología operativa
OWASP	<i>Open Web Application Security Project</i> — proyecto abierto de seguridad en aplicaciones web
RAG	<i>Retrieval-Augmented Generation</i> — generación aumentada por recuperación
RAM	<i>Random-Access Memory</i> — memoria de acceso aleatorio
RF	Requisito funcional
RNF	Requisito no funcional
SCA	<i>Security Configuration Assessment</i> — módulo de evaluación de configuración de seguridad de Wazuh
SIEM	<i>Security Information and Event Management</i> — gestión de información y eventos de seguridad
SOAR	<i>Security Orchestration, Automation and Response</i> — orquestación, automatización y respuesta de seguridad
SOC	<i>Security Operations Center</i> — centro de operaciones de seguridad
SSH	<i>Secure Shell</i> — protocolo de administración remota cifrada
TDP	<i>Thermal Design Power</i> — potencia de diseño térmico
TR-069/TR-369	<i>Technical Report 069/369</i> — protocolos del <i>Broadband Forum</i> para la gestión remota de equipos de cliente
TUI	<i>Text-based User Interface</i> — interfaz de usuario basada en texto
UCI	<i>Unified Configuration Interface</i> — interfaz de configuración unificada de OpenWrt
USB	Universidad Simón Bolívar
VN	Verdadero negativo
VP	Verdadero positivo
WSL	<i>Windows Subsystem for Linux</i> — subsistema de Windows para Linux
XDR	<i>Extended Detection and Response</i> — detección y respuesta extendida
YAML	<i>YAML Ain't Markup Language</i> — formato de serialización de datos legible por personas

Introducción

Los equipos de operaciones de seguridad reciben, en la práctica totalidad de las organizaciones, un volumen de alertas que supera su capacidad de análisis manual. La causa dominante no es la falta de detección, sino el exceso de ella: los sistemas de correlación basados en reglas y firmas —el fundamento de la mayoría de las plataformas de detección y respuesta hoy desplegadas— generan un alto volumen de falsos positivos, y cada uno exige el mismo esfuerzo de interpretación que una amenaza real, porque la alerta señala qué regla se disparó, no por qué eso importa en el contexto concreto del activo afectado.

El presente trabajo, realizado como pasantía larga en la empresa Domotai C.A., aborda ese problema mediante un motor de triaje que clasifica y prioriza alertas de seguridad de red, justifica cada decisión con un modelo de lenguaje de gran escala (LLM, del inglés *Large Language Model*) apoyado en generación aumentada por recuperación (RAG, del inglés *Retrieval-Augmented Generation*) ejecutada en un entorno local, y propone una acción de respuesta contenida sujeta a validación humana antes de ejecutarse.

Antecedentes

Las plataformas comerciales de detección y respuesta han evolucionado del *endpoint* aislado —EDR, del inglés *Endpoint Detection and Response*— hacia la correlación multicapa que combina red, identidad, correo y nube en una sola plataforma, conocida como XDR (*Extended Detection and Response*); y hacia el servicio operado por un equipo humano que las administra las veinticuatro horas, conocido como MDR (*Managed Detection and Response*). Las evaluaciones independientes de estas plataformas —notablemente las de MITRE ATT&CK Evaluations, que mide su cobertura de técnicas adversarias contra la matriz pública MITRE ATT&CK— siguen registrando de forma explícita el volumen de alertas que un analista debe triar como un criterio de comparación entre proveedores, lo que confirma que la fatiga de falsos positivos persiste incluso en las soluciones líderes del mercado.

En paralelo, los modelos de lenguaje de gran escala han empezado a incorporarse a la operación de seguridad. Una primera línea de trabajo continuó el preentrenamiento de mode-

los de tipo *encoder* sobre corpus específicos de ciberseguridad —informes de amenazas, bases de vulnerabilidades, documentación técnica—, de lo que **SecureBERT** es la referencia original [1]. Una segunda línea publicó modelos generativos con preentrenamiento continuado sobre el mismo tipo de corpus, de los que **Foundation-Sec-8B**, de Cisco Foundation AI, es la referencia pública más reciente, con variantes que declaran explícitamente el triaje y el resumen de casos de un centro de operaciones de seguridad (SOC, *Security Operations Center*) como caso de uso objetivo [2]. Las plataformas comerciales líderes —Microsoft Security Copilot, CrowdStrike Charlotte AI y equivalentes— han incorporado asistentes basados en estos modelos, con un patrón consistente: redactan y resumen, no deciden; la autoridad final permanece humana.

Complementando a los modelos generativos, la generación aumentada por recuperación demostró que combinar un modelo de lenguaje con un sistema de recuperación sobre un corpus externo permite anclar sus respuestas a documentos concretos y actualizar ese conocimiento sin reentrenar el modelo [3]. Esta técnica es la que este trabajo aplica para corregir la principal limitación detectada en la justificación generada por un modelo pequeño: describe correctamente qué campos cita, pero se equivoca en lo que esos campos significan.

Justificación e importancia

Adoptar un modelo de lenguaje en el camino de una alerta de seguridad no es una decisión neutra. El catálogo de riesgos de OWASP para aplicaciones basadas en LLM sitúa la inyección de instrucciones como el riesgo número uno [4], y en este dominio no es un riesgo hipotético: el contenido de un registro de eventos —el nombre de usuario probado, la cadena completa del evento— lo escribe quien está atacando. Si esos campos se concatenan al mensaje del modelo sin distinguirlos de una instrucción, el atacante obtiene un canal directo hacia el sistema que decide sobre su propio ataque. A esto se suma un segundo riesgo, más sutil: la alucinación con apariencia de fundamento, en la que el modelo redacta una explicación plausible que cita un dato que la alerta no contiene, y que un analista con prisa acepta precisamente porque suena a análisis correcto.

Estos riesgos, junto con la exigencia de que el análisis de una alerta no salga de la red del cliente hacia un servicio externo, justifican dos decisiones de diseño que atraviesan todo el trabajo: que los campos de una alerta se traten siempre como datos y nunca como instrucciones, y que el modelo se ejecute en un entorno local, sin depender de una nube de terceros. La segunda decisión responde además a una necesidad de negocio concreta de la empresa: Domotai C.A. ofrece la automatización asistida por inteligencia artificial como una de sus líneas de servicio, y un módulo que reduzca el ruido operativo de un cliente sin exponer sus datos ocupa un lugar dentro de esa oferta que las plataformas comerciales, todas ellas

dependientes de la nube del proveedor, no cubren.

Por último, la restricción de continuidad de servicio es la que distingue a este trabajo de un sistema de respuesta automatizada convencional: un triaje que acierta más pero que, por error, interrumpe un servicio que el cliente presta a los suyos es peor que el sistema al que sustituye. De ahí que la decisión de qué acción ejecutar nunca la tome el propio modelo de lenguaje, sino una política determinista filtrada por un perfil de cliente configurable, con la validación de una persona antes de cualquier acción que alcance un servicio.

Planteamiento del problema

Diseñar este motor de triaje sobre el sistema de análisis de alertas que la empresa usa hoy exigía partir de tres elementos que, en la práctica, no estaban disponibles al inicio de la pasantía: el propio sistema de logs y correlación de un cliente real, un entorno donde ejercitarlo sin riesgo para una operación productiva, y un conjunto de alertas con verdaderos y falsos positivos identificados contra los que medir cualquier mejora. Sin una fuente de alertas no hay nada que triar; sin un entorno controlado no hay dónde ejecutar ni verificar una acción de respuesta; y sin un conjunto etiquetado no es posible calcular precisión, exhaustividad ni ninguna otra métrica de clasificación.

El problema, formulado de manera acotada, es entonces el siguiente: dado que no se dispone de acceso a un cliente ni a su módulo propietario de análisis de alertas, ¿es posible construir, evaluar y demostrar en un entorno de laboratorio reproducible un motor de triaje que clasifique alertas de acceso no autorizado a servicios de gestión expuestos, justifique cada decisión con razonamiento verificable en vez de con la sola cita de una regla, y proponga una acción de respuesta acotada sin comprometer la continuidad del servicio que protege, de forma que el resultado sea representativo de lo que ese motor haría sobre la infraestructura real de un cliente cuando esta esté disponible?

La respuesta de diseño fue modelar un cliente genérico en vez de esperar a uno concreto, y sustituir el sistema de logs inexistente por Wazuh, una plataforma de detección basada en reglas desplegada dentro de un laboratorio de red emulado con Containerlab [5], cuyo nivel de regla cumple además el papel de método tradicional de referencia contra el que se mide el prototipo.

Objetivo general

Desarrollar un prototipo para la detección y el triaje de alertas de seguridad en entornos empresariales, basado en modelos de lenguaje con un módulo de generación aumentada por recuperación ejecutado sobre un modelo desplegado de forma local, de manera que permita

clasificar y priorizar alertas con razonamiento explicable, reduciendo los falsos positivos y apoyando la labor de los analistas de seguridad.

Objetivos específicos

1. **Analizar y aprender a usar el módulo propietario.** Analizar la solución actual de la empresa para la recepción y el análisis de alertas, identificar sus limitaciones y las necesidades no cubiertas, y definir el caso de uso acotado sobre el que se construiría el prototipo.
2. **Revisar el estado del arte.** Analizar las soluciones de detección y respuesta extendida y gestionada existentes en el mercado y las aplicaciones de modelos de lenguaje en el ámbito de la ciberseguridad, identificando sus ventajas, sus limitaciones y las necesidades no cubiertas por las herramientas actuales, para derivar de ello los requisitos funcionales y no funcionales del prototipo.
3. **Configurar un entorno de pruebas.** Instalar y poner en funcionamiento un entorno con casos de prueba representativos, garantizando que los datos no salgan hacia servicios externos, y preparar un conjunto de alertas de prueba etiquetado —verdaderos y falsos positivos— que sirviera de referencia de evaluación.
4. **Diseñar la arquitectura del sistema.** Definir el flujo de procesamiento, las reglas de decisión y sus umbrales, la forma de interacción con el entorno de pruebas, el formato del razonamiento explicable y los puntos de validación humana, así como las métricas de evaluación y el método tradicional de referencia contra el que se compararía el prototipo.
5. **Implementar el prototipo.** Construir el caso de uso acotado, integrando la ingesta y normalización de alertas, la clasificación y priorización, la generación de justificación explicable y el flujo de validación humana en las decisiones críticas, con registro de trazas para auditoría.
6. **Evaluar el prototipo.** Medir el desempeño del sistema frente al método tradicional de referencia definido en el objetivo anterior, y analizar los errores sistemáticos y las limitaciones del enfoque.
7. **Elaborar la documentación técnica.** Generar la documentación y el informe técnico con la arquitectura, las decisiones de diseño, los resultados, las limitaciones y las líneas de trabajo futuro.

Estructura del informe

El documento se organiza en cuatro capítulos. El primero describe la empresa donde se desarrolló la pasantía. El segundo reúne el marco teórico, recorriendo desde el problema del triaje de alertas y las plataformas de detección y respuesta hasta los modelos de lenguaje, la generación aumentada por recuperación, los riesgos de aplicarla a este dominio y las métricas con que se evalúa. El tercero expone la metodología seguida a lo largo de las seis fases del proyecto, y el cuarto presenta los resultados obtenidos y su discusión. Cierran el documento las conclusiones y recomendaciones, y las referencias consultadas.

CAPÍTULO 1

DESCRIPCIÓN DE LA EMPRESA

La pasantía se desarrolla en Domotai C.A., una empresa venezolana de desarrollo de software enfocada en la aplicación de la inteligencia artificial a productos y procesos. Este capítulo describe su actividad, sus líneas de servicio y su forma de trabajo, y sitúa el proyecto de la pasantía dentro de ese contexto.

1.1. Identificación y ubicación

Domotai C.A. opera desde el Edificio Caracas Campus, en la calle Altagracia, urbanización La Trinidad, municipio Baruta del estado Miranda, en la ciudad de Caracas. Su actividad no se limita al mercado local: atiende clientes de sectores diversos bajo un modelo de trabajo que no exige la presencia física del cliente en sus instalaciones.

1.2. Actividad y servicios

La empresa combina el criterio de sus profesionales con herramientas de inteligencia artificial para el desarrollo de productos de software. Su oferta se organiza en las líneas recogidas en la tabla 1.1.

Tabla 1.1. Líneas de servicio de Domotai C.A.

Línea	Alcance
Desarrollo web	Sitios corporativos y aplicaciones de cliente desarrollados a medida
Desarrollo móvil	Aplicaciones multiplataforma
Automatización con inteligencia artificial	Agentes y flujos automatizados integrados en los procesos del cliente
Integración <i>hardware-software</i>	Soluciones que articulan componentes físicos con sistemas informáticos
Diseño de interfaz y experiencia de usuario	Definición de la interacción y del aspecto visual de los productos
Asesoría tecnológica	Acompañamiento en decisiones de arquitectura y selección de herramientas

De estas líneas, la automatización mediante inteligencia artificial es la que enmarca directamente el proyecto de esta pasantía y a la que se vuelve al cierre del capítulo.

1.3. Modelo de trabajo

Domotai C.A. acumula más de veinte proyectos completados, con un plazo de entrega característico del orden de seis semanas: un modo de operar orientado a ciclos cortos, en los que el alcance del producto se acota para poder entregarse dentro de ese plazo en lugar de plantearse como un desarrollo prolongado. Su cartera abarca dominios heterogéneos —diseño y arte, hostelería, interiorismo, comercio electrónico—, lo que indica que su especialización no reside en un sector vertical concreto, sino en un conjunto de capacidades técnicas aplicables a dominios distintos.

La contratación se articula bajo dos modalidades: por proyecto, con un alcance cerrado de antemano, y por suscripción, que da al cliente acceso continuado a la capacidad de desarrollo de la empresa. La segunda modalidad supone una relación sostenida en el tiempo, lo que favorece la acumulación de conocimiento sobre el dominio de cada cliente.

1.4. Ámbito de la pasantía

El proyecto desarrollado durante esta pasantía se sitúa en la línea de automatización con inteligencia artificial descrita en la sección 1.1, aplicada esta vez al dominio de la cibersegu-

ridad: un motor que triaje alertas de seguridad de red, las clasifique y priorice, y justifique cada decisión mediante un modelo de lenguaje con generación aumentada por recuperación, en lugar de mediante trabajo manual especializado de un analista.

El encaje con la empresa es directo por dos razones. La primera es de mercado: Domotai C.A. no compite en un segmento vertical fijo, sino que ofrece capacidades de automatización aplicables a dominios distintos, y la seguridad de red —con su necesidad de reducir el trabajo manual repetitivo de un especialista— es uno de esos dominios. La segunda es de restricción compartida: al igual que en el resto de su cartera, la solución debía operar sin depender de servicios en la nube de terceros, de forma que los datos del cliente —en este caso, sus alertas de seguridad— no salgan de su propia red.

La supervisión de la pasantía corresponde a un tutor industrial de la compañía, cuyo nombre y cargo quedan por confirmar en la versión definitiva de este documento. La estancia se desarrolla bajo la modalidad y la duración correspondientes a una pasantía larga de la Universidad Simón Bolívar.

CAPÍTULO 2

MARCO TEÓRICO

Este capítulo reúne los fundamentos sobre los que se apoya el desarrollo. Avanza desde el problema que motiva el proyecto —la fatiga de falsos positivos en el triaje de alertas— hacia las plataformas que hoy lo abordan, los modelos de lenguaje que este trabajo incorpora, la técnica de recuperación que corrige su principal debilidad, los riesgos de aplicarlos a este dominio, el razonamiento que sostiene que la decisión final no debe tomarla el modelo, y por último los criterios con que se mide el resultado.

2.1. El problema del triaje de alertas de seguridad

Un sistema de detección basado en reglas y firmas dispara una alerta cuando una secuencia de eventos coincide con un patrón previamente definido: una autenticación fallida, un puerto que empieza a recibir tráfico, un proceso que no debería ejecutarse. Esa coincidencia es determinista y auditable —se puede señalar exactamente qué regla se disparó—, pero no incorpora ningún juicio sobre si, en el contexto del activo concreto que la generó, esa coincidencia representa una amenaza real. La misma regla se dispara igual sobre un servidor expuesto a internet que sobre uno que no lo está, y el resultado es un volumen de falsos positivos que crece con el tamaño de la red sin que crezca en la misma proporción el riesgo real.

El *traje* es la tarea de decidir, para cada alerta, si merece la atención de un analista y con qué prioridad. Cuando esa tarea se realiza manualmente sobre un volumen de alertas que supera la capacidad del equipo, dos fallos son posibles y ambos son costosos: tratar como amenaza algo que no lo es, que consume tiempo del analista sin reducir el riesgo, y descartar como ruido algo que sí lo es, que es exactamente el fallo que un sistema de seguridad no puede permitirse. Las evaluaciones independientes de las plataformas comerciales de detección más recientes miden de forma explícita el volumen de alertas que un analista debe procesar como un criterio de comparación entre proveedores [6], lo que confirma que este problema no es una limitación de las herramientas más antiguas, sino una característica persistente del propio enfoque basado en reglas, incluso en sus implementaciones más avanzadas.

2.2. Detección y respuesta: EDR, XDR y MDR

Tres siglas dominan el mercado de plataformas de detección y conviene distinguirlas, porque operan en planos distintos.

EDR (*Endpoint Detection and Response*, detección y respuesta en el *endpoint*) es la tecnología que recoge telemetría de un equipo concreto —procesos, ficheros, registro, conexiones de red locales— y responde sobre ese mismo equipo: aislarlo, terminar un proceso, ponerlo en cuarentena.

XDR (*Extended Detection and Response*, detección y respuesta extendida) correlaciona telemetría de varias capas —*endpoint*, red, identidad, correo, nube— en una sola plataforma analítica. El EDR es, en este esquema, una de las fuentes que alimentan al XDR; lo que este añade es la correlación entre fuentes que, por separado, solo verían una parte del incidente.

MDR (*Managed Detection and Response*, detección y respuesta gestionada) no es una tecnología sino un servicio: un equipo humano que opera un EDR o un XDR por cuenta del cliente, haciendo el triaje, la investigación y la respuesta que el cliente no tiene capacidad de operar por sí mismo. Un MDR que opera sobre un XDR se denomina en ocasiones MXDR (*Managed XDR*).

Las plataformas comerciales líderes despliegan típicamente un agente propietario a nivel de sistema operativo, más rico que las fuentes de telemetría de código abierto —como Sysmon, osquery o Zeek— y con capacidad de respuesta activa integrada. Esas fuentes abiertas, sin embargo, no quedan excluidas del ecosistema: numerosas plataformas las ingieren como fuente complementaria en su capa de agregación, y los servicios MDR que no exigen un agente comercial concreto —el modelo que en este trabajo se denomina telemetría-agnóstico— operan directamente sobre la telemetría que el cliente ya tenga. Este proyecto se inserta precisamente en ese segundo modelo: no sustituye al agente ni a la fuente de telemetría, sino que añade la capa de clasificación, priorización y justificación explicable sobre las alertas que esa fuente ya produce.

2.3. MITRE ATT&CK como taxonomía de referencia

MITRE ATT&CK (*Adversarial Tactics, Techniques, and Common Knowledge*) es una matriz pública y de acceso libre que cataloga las tácticas y técnicas que un atacante emplea a lo largo de un incidente, desde el acceso inicial hasta la exfiltración, con un identificador único por técnica —por ejemplo, T1110 para el acceso por fuerza bruta, con la subtécnica T1110.001 para el intento por diccionario de contraseñas [7]. Su valor para este trabajo es doble: sirve como vocabulario común para expresar la hipótesis de ataque dentro de la justificación de

cada decisión, y sirve como criterio de comparación en las evaluaciones independientes de plataformas comerciales citadas en la sección anterior. MITRE mantiene además ATLAS, una matriz equivalente centrada en las amenazas contra los propios sistemas de inteligencia artificial [8], pertinente en la medida en que este trabajo introduce uno de esos sistemas en el propio camino de decisión.

2.4. Modelos de lenguaje de gran escala aplicados a la seguridad

Un modelo de lenguaje de gran escala (LLM) es una red neuronal entrenada para predecir la continuación más probable de una secuencia de texto, sobre un volumen de parámetros que va de varios cientos de millones a varios cientos de miles de millones. Aplicado a la seguridad, conviene distinguir tres formas de uso, que no compiten entre sí sino que resuelven partes distintas del problema de triaje:

- **Codificador clasificador** (*encoder*, con arquitectura de tipo BERT y del orden de cien millones de parámetros): asigna una clase y una confianza numérica a una entrada, tras un ajuste fino sobre datos etiquetados. Su coste de ejecución es muy bajo —del orden de milisegundos sobre un procesador convencional— y es determinista, pero no redacta ninguna explicación y no generaliza bien fuera de lo que vio durante el entrenamiento.
- **Modelo generativo instruido** (de varios miles de millones de parámetros): redacta el razonamiento, resume un incidente o traduce una consulta en lenguaje natural. Su coste es alto —segundos y varios gigabytes de memoria— y no es fiable como clasificador numérico ni determinista por defecto.
- **Modelo especializado de dominio**: un codificador o un generativo con preentrenamiento continuado sobre corpus de ciberseguridad. **SecureBERT** demostró que esa especialización mejora las tareas de clasificación del dominio frente al modelo genérico equivalente [1], y **Foundation-Sec-8B**, un modelo generativo de ocho mil millones de parámetros con preentrenamiento continuado sobre corpus de inteligencia de amenazas, declara explícitamente el triaje de un centro de operaciones de seguridad como uno de sus casos de uso objetivo [2]. La especialización aporta vocabulario —el modelo reconoce sin explicación previa un identificador de vulnerabilidad o una técnica ATT&CK— pero no aporta garantías: sigue pudiendo afirmar con aparente seguridad algo que la entrada no contiene.

La conclusión que este marco teórico traslada al diseño del prototipo es que clasificar y justificar son tareas de naturaleza distinta y conviene resolverlas con componentes distintos:

un codificador da una clase estable, barata y con una confianza calibrable; un modelo generativo produce el texto que una persona necesita leer para decidir. Pedirle ambas cosas al mismo componente tiende a producir puntuaciones de confianza inventadas con apariencia de cálculo.

2.5. Generación aumentada por recuperación

La generación aumentada por recuperación (RAG) combina un modelo de lenguaje con un mecanismo de recuperación de información sobre un corpus documental externo: ante una consulta, el sistema recupera primero los fragmentos de texto más relevantes de ese corpus y se los entrega al modelo como parte de su contexto, antes de que este redacte la respuesta [3]. Dos consecuencias hacen de esta técnica algo más que un atajo de ingeniería. La primera es que el conocimiento deja de residir exclusivamente en los parámetros del modelo —fijados en el momento de su entrenamiento— y pasa a un índice que puede actualizarse sin reentrenar nada. La segunda es que la respuesta queda *anclada*: cada afirmación puede, en principio, rastrearse hasta el fragmento del corpus que la sustenta, lo que reduce —sin eliminar— el riesgo de que el modelo invente un dato plausible.

La forma más simple de recuperación —la empleada en este trabajo— representa cada documento del corpus y cada consulta como un vector numérico mediante un modelo de *embeddings*, y ordena los documentos por su similitud coseno con la consulta:

$$\cos(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|} \quad (2.1)$$

donde $\mathbf{q}$ es el vector de la consulta y $\mathbf{d}$ el de un documento del corpus; se seleccionan los k documentos de mayor similitud, con k típicamente pequeño (en este trabajo, $k = 3$) para no saturar el contexto del modelo con pasajes de relevancia marginal. Existen técnicas de recuperación más sofisticadas —dispersas, como BM25, o densas entrenadas específicamente para la tarea de recuperación, como en los sistemas de tipo *dense passage retrieval*— así como arquitecturas de recuperación por grafo; este trabajo emplea la variante más simple posible, por similitud de *embeddings* genéricos, dado que el objetivo no es optimizar la recuperación en sí sino verificar si corregir el conocimiento del modelo mejora la corrección de su justificación.

Un riesgo documentado de esta técnica, especialmente relevante cuando la consulta y el corpus se formulan en idiomas o registros distintos, es que la utilidad de un fragmento recuperado no es uniforme según su posición en el contexto que finalmente recibe el modelo: los fragmentos situados en el centro de un contexto largo se aprovechan peor que los situados en sus extremos [9]. Esta observación motivó, en el diseño del justificador de este trabajo,

mantener el número de pasajes recuperados deliberadamente bajo.

2.6. Riesgos de aplicar un modelo de lenguaje al triaje de seguridad

Introducir un modelo de lenguaje en el camino de decisión de una alerta de seguridad tiene riesgos específicos de este dominio, más allá de los riesgos generales de cualquier aplicación basada en LLM catalogados por el proyecto abierto de seguridad en aplicaciones web [4] y por la evaluación de seguridad de modelos de Meta conocida como CyberSecEval [10].

Inyección de instrucciones desde campos controlados por el atacante. El contenido de un registro de eventos —el nombre de usuario probado, la cadena de *user-agent*, el mensaje completo del evento— lo escribe, en un intento de intrusión, la propia parte atacante. Si esos campos se concatenan sin distinguirlos de las instrucciones del sistema, el atacante dispone de un canal directo hacia el componente que analiza su propio ataque. La respuesta de diseño es tratar siempre esos campos como datos delimitados, nunca como instrucciones.

Alucinación con apariencia de fundamento. El fallo más peligroso no es el error evidente, sino la explicación bien redactada que cita un detalle que la entrada no contiene; un analista con prisa la acepta precisamente porque suena a análisis correcto. El contrapeso es exigir un anclaje verificable: toda afirmación debe poder rastrearse a un campo concreto de la alerta o del contexto recuperado, y esa verificación puede automatizarse.

Ausencia de determinismo. La misma entrada puede producir salidas distintas en dos ejecuciones sucesivas de un modelo generativo. Para la operación es una incomodidad menor; para la evaluación es descalificante, porque impide que los resultados sean reproducibles. Se controla fijando una temperatura de muestreo próxima a cero y registrando la versión exacta del modelo y de las instrucciones empleadas en cada ejecución.

Corte de conocimiento. Un modelo no sabe nada de lo publicado después de su entrenamiento, así que cualquier afirmación suya sobre el panorama de amenazas actual es sospechosa por construcción; el conocimiento reciente debe entrar por el contexto que se le suministra, no salir de sus propios parámetros.

Superficie de fallo adicional. Un componente más que puede fallar, colgarse o rendir por debajo de lo esperado exige un mecanismo de degradación: si el modelo no responde o su respuesta no pasa la verificación de anclaje, la decisión debe seguir un camino alternativo —una plantilla determinista, en este trabajo— y nunca descartarse en silencio.

2.7. Reglas y firmas frente a análisis asistido por modelos de lenguaje

La tabla 2.1 resume la comparación entre ambos enfoques sobre los atributos que importan para el triaje.

Tabla 2.1. Reglas y firmas frente a análisis asistido por modelos de lenguaje

	Reglas y firmas	Análisis asistido por modelo
Decisión	Determinista y repetible	Probabilístico; repetible solo si se fuerza
Auditabilidad	Total: se lee la regla que se disparó	Indirecta: se lee la justificación, no el cálculo
Contexto del activo	Ninguno	Su mejor aportación, si se le suministra
Explicación	El nombre de la regla	Razonamiento en lenguaje natural
Generalización	Nula ante lo no previsto	Parcial: reconoce variantes de lo ya visto
Fallo típico	Falso positivo por falta de contexto	Afirmación segura y equivocada

Ninguno de los dos enfoques sustituye por completo al otro: las reglas son buenas detectando y malas decidiendo qué importa, y el modelo es débil detectando por sí solo pero fuerte explicando y priorizando lo que ya fue detectado. El diseño que este trabajo adopta usa las reglas como fuente —de ahí que Wazuh se conserve como origen de las alertas— y el modelo como capa de triaje sobre ellas, nunca al revés.

2.8. Políticas de decisión determinista y separación entre analizar y actuar

Un motor de triaje que, además de clasificar una alerta, decidiera con el mismo componente generativo qué acción de respuesta ejecutar, heredaría todos los riesgos de la sección anterior en el punto de mayor consecuencia del sistema: el momento de actuar sobre la infraestructura del cliente. La alternativa que sostiene el diseño de este trabajo separa esa decisión en dos capas con motivos de cambio distintos: una *política* determinista, expresada como una tabla indexada por la clase de la alerta y el contexto del activo, que *propone* la acción de menor impacto capaz de resolver la amenaza; y un *perfil de cliente* configurable, que *filtra* esa propuesta según lo que el negocio del cliente concreto permite, degradándola a una de menor

impacto o reteniéndola para validación humana. El modelo de lenguaje interviene únicamente para redactar la justificación de una decisión ya tomada por la política, nunca para tomarla. Esta separación es, en sí misma, una aplicación directa del principio de auditabilidad: un error se puede atribuir a la clasificación, a la política o al perfil, y se sabe cuál de los tres corregir, algo que no sería posible si toda la decisión residiera dentro de un único componente generativo.

2.9. Métricas de evaluación de un clasificador binario

Evaluar si el triaje asistido mejora sobre el método de referencia exige medir su desempeño como clasificador binario —amenaza o no amenaza— sobre una matriz de confusión con cuatro categorías: verdadero positivo (VP), cuando el sistema marca como amenaza una alerta que efectivamente lo es; falso positivo (FP), cuando la marca como amenaza sin serlo; verdadero negativo (VN), cuando correctamente no la marca; y falso negativo (FN), cuando una amenaza real pasa desapercibida. De esa matriz se derivan las métricas de la tabla 2.2.

Tabla 2.2. Métricas de clasificación empleadas en la evaluación

Métrica	Fórmula	Qué mide
Precisión (P)	$\frac{VP}{VP + FP}$	De lo marcado como amenaza, cuánto lo era realmente
Exhaustividad (R)	$\frac{VP}{VP + FN}$	De las amenazas reales, cuántas se detectaron
Medida F_1	$\frac{2PR}{P + R}$	Equilibrio entre precisión y exhaustividad
Tasa de falsos positivos	$\frac{FP}{FP + VN}$	El ruido operativo que sufre el analista

De estas cuatro, la tasa de falsos positivos es la que responde más directamente al problema planteado en la sección 2.1: mide exactamente la proporción de ruido que un analista tendría que descartar manualmente. La exhaustividad, por su parte, es la métrica que no puede sacrificarse bajo ninguna circunstancia: un sistema que reduce el ruido a costa de dejar pasar amenazas reales no es una mejora, sino un riesgo nuevo.

2.10. Entornos de red virtualizados para experimentación en seguridad

Ejercitar y medir un motor de triaje exige un entorno donde generar actividad legítima y hostil sobre una red con vulnerabilidades conocidas, sin exponer una infraestructura productiva. Las plataformas de este tipo se dividen en dos familias con una diferencia determinante para este trabajo: las que *simulan* un dispositivo —reproducen su comportamiento observable sin ejecutar su sistema operativo real— y las que lo *virtualizan* o lo *emulan*, ejecutando una imagen real del sistema dentro de un contenedor o una máquina virtual. Solo la segunda familia expone servicios genuinos que una herramienta de escaneo de vulnerabilidades pueda encontrar, lo que descarta a los simuladores puramente didácticos como plataforma de laboratorio para este proyecto.

Containerlab es una de las plataformas de esa segunda familia: define la topología de red como un fichero de texto versionable, orquesta contenedores y máquinas virtuales ligeras según ese fichero, y crea automáticamente una red de gestión fuera de banda que conecta todos los nodos entre sí [5]. Su reproducibilidad —una topología descrita en un fichero puede destruirse y recrearse de forma idéntica— es lo que hace posible que las condiciones de una campaña de evaluación puedan repetirse de manera fiel, un requisito directo de la reproducibilidad exigida a la propia evaluación del capítulo 3.

CAPÍTULO 3

METODOLOGÍA

El trabajo se organizó en seis fases sucesivas, correspondientes una a una con los objetivos específicos enunciados en la Introducción, más una séptima fase de consolidación documental. Cada fase produjo un entregable concreto sobre el que se apoya la siguiente, y las decisiones de diseño que no podían anticiparse al inicio —porque dependían de una medición, no de un criterio— se tomaron y registraron en el momento en que la medición estuvo disponible, con su fecha. Esta sección documenta esas decisiones en el orden en que se tomaron.

3.1. Enfoque general: modelar en lugar de esperar

La primera decisión metodológica, y la que condiciona a todas las demás, se tomó antes de escribir ninguna línea de código: al no disponer de acceso a un cliente real ni a su sistema propietario de análisis de alertas, el trabajo optó por *modelar* un cliente genérico en lugar de *analizar* uno concreto. Analizar un cliente real habría producido un prototipo ajustado a ese cliente; modelar uno genérico obligó a separar explícitamente, desde el principio, lo que cualquier empresa comparte —una infraestructura de red organizada por capas, con una superficie de gestión remota presente en todas ellas— de lo que varía de una a otra —su catálogo de acciones permitidas, su política de continuidad de servicio, la criticidad de sus activos—. Esa separación es la que permite que el prototipo resultante sea configurable por cliente sin cambios de código, y es la misma lógica que, más adelante, resolvió la ausencia de una fuente real de alertas: en vez de esperar a que un sistema propietario estuviera disponible, se sustituyó por una plataforma de detección basada en reglas —Wazuh— desplegada dentro de un laboratorio propio, con la ventaja adicional de que su nivel de regla podía servir directamente como método tradicional de referencia.

3.2. Fase 1: modelado del cliente y del caso de uso

Se construyó un modelo de cliente genérico describiendo su infraestructura de red por capas —punto de entrega, equipo de borde, conmutación, zona desmilitarizada, servidores

internos, puestos de trabajo, dispositivos IoT y plano de gestión—, identificando en cada una qué expone hacia la administración remota y qué ocurre si se interrumpe. De ese modelo se derivaron cinco puntos de variabilidad configurables entre clientes —el esquema de la alerta, el repertorio de acciones y su canal, la política de continuidad, el inventario y criticidad de activos, y la severidad de referencia— y un conjunto de restricciones operativas que no son negociables en ningún cliente: que ningún servicio se detenga sin autorización, que no exista una ventana de mantenimiento sobre la que apoyarse, que toda acción sea reversible porque no hay manos físicas junto al equipo, y que ninguna acción pueda cortar el propio canal de gestión por el que se administra y se responde.

Sobre ese modelo se acotó el caso de uso a una sola familia de alertas —el acceso no autorizado a servicios de gestión expuestos—, deliberadamente definida por el tipo de patrón y no por el equipo concreto que lo sufre: la misma fuerza bruta contra un servicio de administración se reconoce igual en un conmutador que en un servidor de ficheros, lo que permite que el sistema generalice al parque de equipos de cualquier cliente en lugar de aprender las particularidades de uno solo.

3.3. Fase 2: estado del arte y registro de requisitos

Se revisaron, por un lado, las plataformas comerciales de detección y respuesta extendida y gestionada y sus limitaciones documentadas, y por otro, las aplicaciones de modelos de lenguaje a la seguridad, incluyendo sus riesgos específicos. De ese análisis se derivó un registro único de veinte requisitos funcionales y catorce no funcionales, cada uno trazado hasta la limitación de mercado o el criterio de diseño que lo motiva. Cuando, más adelante, el encuadre del proyecto se desplazó del segmento de pequeña y mediana empresa hacia la red de un cliente modelado en la Fase 1, ese registro se revisó íntegramente contra el nuevo contexto: ningún requisito original dejó de aplicar, seis cambiaron de matiz y siete se añadieron —principalmente los que más adelante formalizarían la política de continuidad de servicio—, lo que se documentó como una revisión explícita y no como una reescritura silenciosa.

3.4. Fase 3: construcción del entorno de pruebas

El entorno de pruebas se construyó como una red de cliente emulada con Containerlab, reproduciendo desde el punto de entrega del proveedor hacia dentro: un equipo de borde, un conmutador de red interna, un puesto de usuario, un dispositivo de tipo IoT con servicios expuestos y un servidor con vulnerabilidades documentadas —Metasploitable2, con diecinueve puertos abiertos y fallos conocidos y públicos—, todos conectados además a una red de gestión fuera de banda por la que operan, sin contaminar el tráfico observado, el motor de

triaje y un nodo auditor con Nmap.

Un mismo laboratorio debía resolver dos necesidades que, se verificó explícitamente, no son intercambiables: un *ground truth* de vulnerabilidades —qué falla exactamente en cada nodo— y un *ground truth* de alertas —si una alerta concreta es un verdadero o un falso positivo—. El primero sale directamente de la composición documentada de la topología; el segundo exigió un puente explícito: cada alerta que Wazuh generaba sobre la actividad del laboratorio se contrastaba contra el inventario de vulnerabilidades del nodo al que apuntaba, y de esa comparación —con revisión manual, no completamente automatizable— salía la etiqueta de verdadero o falso positivo, conservando en todo momento el nivel de regla original de Wazuh como el dato que más adelante serviría de baseline.

El resultado, cerrado el 31 de agosto de 2026, fue un conjunto de 410 alertas reales generadas sobre el propio laboratorio, etiquetadas y particionadas por campaña: 20 verdaderos positivos, 16 falsos positivos y 374 alertas de ruido de plataforma —telemetría de evaluación de configuración que no corresponde a ningún patrón del caso de uso acotado—, todas ellas concentradas, las soportadas, en un único activo y servicio (acceso por SSH al servidor con vulnerabilidades documentadas). Esta concentración en una sola familia de ataque, verificada al construir el conjunto y no descubierta después, es la que más adelante bloquearía el ajuste fino del clasificador y limitaría el alcance estadístico de la evaluación del capítulo 4: se documenta aquí como una condición conocida del método, no como un hallazgo posterior.

3.5. Fase 4: diseño de la arquitectura

El diseño se apoyó en tres puntos de aislamiento deliberados, cada uno resuelto mediante un adaptador que esconde una dependencia externa hoy provisional: un módulo de ingesta que aísla la fuente de alertas concreta —hoy Wazuh, mañana el sistema propietario del cliente—; una interfaz de análisis de dos operaciones, *clasificar* y *justificar*, que aísla el modelo de lenguaje concreto detrás de cada una; y un conector que aísla el protocolo de comunicación hacia el equipo objetivo —hoy SSH sobre un catálogo cerrado de acciones—. Ninguno de los tres exige reescribir la lógica de decisión del motor de triaje cuando la pieza que ocultan cambie.

Sobre esa interfaz de análisis se definieron dos perfiles de despliegue según el hardware disponible: un perfil híbrido para el equipo de desarrollo entonces disponible —un procesador de propósito general de ocho núcleos, sin GPU dedicada y con dieciséis gigabytes de memoria—, que combina un codificador ligero para clasificar con un modelo generativo pequeño para justificar en línea; y un perfil de un único componente generativo especializado, viable únicamente con una GPU dedicada o memoria sustancialmente mayor. La decisión ini-

cial del 24 de agosto de 2026 estimaba, sobre mediciones publicadas en hardware comparable, que un modelo generativo de tres mil ochocientos millones de parámetros cuantizado rendiría entre diez y doce *tokens* por segundo en el equipo disponible. La medición directa realizada el 2 de septiembre de 2026 dio un resultado muy distinto —aproximadamente 3,7 *tokens* por segundo—, lo que se documentó de inmediato como una corrección y no se ocultó: con ese rendimiento, un modelo de ese tamaño quedaba fuera del presupuesto de latencia tolerable para una validación humana interactiva, y el justificador se implementó finalmente sobre un modelo generativo de mil millones de parámetros. La interfaz de análisis, precisamente por estar aislada del modelo concreto, no tuvo que rediseñarse para absorber ese cambio.

También en la Fase 4 se resolvió que la decisión de qué acción proponer nunca recae en el modelo de lenguaje, por las razones expuestas en la sección 2.8 del marco teórico: una tabla de política determinista, indexada por la clase de la alerta, propone la acción de menor impacto que resuelve la amenaza, y un perfil de cliente configurable la filtra —permitiéndola, degradándola a una de menor impacto o reteniéndola para validación humana— según lo que ese cliente concreto permite. Se definió además el plan de métricas de la evaluación —precisión, exhaustividad, medida F_1 , tasa de falsos positivos, acierto de prioridad y dos métricas de continuidad específicas de este caso de uso: acciones disruptivas indebidas y retención correcta—, y se fijó como método tradicional de referencia el nivel de regla de Wazuh, por ser literalmente el enfoque basado en reglas y firmas contra el que el proyecto se planteó comparar.

3.6. Fase 5: implementación

El prototipo se implementó en Python 3, empleando únicamente su biblioteca estándar más `pyyaml` para la configuración, sin gestor de paquetes ni entorno virtual adicional para el motor de decisión, y con `unittest` como marco de pruebas. La implementación se dividió en tres subproyectos:

- **Núcleo de decisión.** La cadena ingesta → análisis → política → perfil → traza, ejecutable en lote sobre el conjunto de alertas etiquetado, con un clasificador de referencia determinista —una regla fija sobre la postura de exposición del activo, sin aprendizaje— detrás de la interfaz `clasificar`. Este clasificador se mantiene como una pieza provisional deliberada: el ajuste fino de un codificador quedó bloqueado precisamente por la concentración en una sola familia de ataque documentada en la Fase 3, sin variedad suficiente de clases para entrenar y validar un modelo que generalizara.
- **Lazo en vivo.** Un conector que traduce cada acción abstracta del catálogo cerrado a un comando concreto sobre el nodo objetivo por SSH, con verificación de idempotencia

previa a la ejecución, validación de los parámetros contra caracteres de control de shell, y una segunda verificación posterior para confirmar el efecto; una validación humana por terminal que muestra la alerta, la clase, la prioridad, la justificación y la acción propuesta antes de ejecutarla; y el registro de traza auditable de cada decisión. El ejecutor SSH real se inyecta como una función sustituible, de forma que la lógica de idempotencia y de verificación se pudo probar con un ejecutor simulado sin necesidad del laboratorio activo.

- **Justificador con modelo de lenguaje real.** Un modelo generativo de mil millones de parámetros, cuantizado a cuatro bits, invocado mediante un proceso independiente a un motor de inferencia en un entorno Conda separado del resto del prototipo. Cada justificación se somete a una verificación de anclaje automática —ninguna dirección IP mencionada puede ser distinta de la de la alerta, y el texto debe referenciar al menos un campo concreto de esta— y, si no la supera o el modelo falla, se degrada a una plantilla determinista que por construcción sí ancla, de forma que ninguna alerta queda nunca sin justificación.

Sobre esa misma interfaz se añadió un cuarto componente, no previsto en el diseño original de la Fase 4 sino incorporado tras detectar en la Fase 6 que el justificador de mil millones de parámetros, aunque anclaba correctamente sus afirmaciones, las describía con un significado equivocado: un módulo de generación aumentada por recuperación que indexa, mediante un modelo de *embeddings* del mismo entorno de inferencia, un corpus curado de fichas sobre las técnicas MITRE ATT&CK, las reglas de Wazuh y las vulnerabilidades del laboratorio pertinentes al caso de uso, y recupera los tres pasajes más afines a cada alerta —por similitud coseno, según la ecuación de la sección 2.1 del marco teórico— antes de invocar al modelo generativo. Dos decisiones de diseño acotan deliberadamente su alcance: la consulta de recuperación se construye únicamente a partir de campos estructurados de la alerta —el identificador de la regla, la técnica y el servicio—, nunca del registro completo del evento, por la misma razón de seguridad que ya impedía tratar ese campo como instrucción; y el corpus se mantiene reducido y curado a mano, en vez de indexar documentación completa, porque el objetivo era verificar si corregir el conocimiento del modelo mejoraba la corrección de su justificación, no maximizar la cobertura de la recuperación.

Todo el prototipo se validó con una batería de pruebas unitarias que, en el momento de escribir este informe, asciende a más de un centenar de casos, ejecutados con generadores y ejecutores simulados que permiten probar la lógica de decisión, de idempotencia y de degradación sin necesidad de un modelo de lenguaje cargado ni de un laboratorio en ejecución.

3.7. Fase 6: evaluación

La evaluación se diseñó como una campaña reproducible por línea de comandos que procesa una partición del conjunto de alertas etiquetado, calcula la clasificación del prototipo y, sobre la misma partición, calcula el mejor punto de un barrido del nivel de regla de Wazuh como umbral de clasificación —el método tradicional de referencia—, de forma que ambos se comparan siempre bajo las mismas condiciones y contra el mismo *ground truth*. Para la calidad de la justificación se definieron dos evaluaciones complementarias: una automática, que mide la tasa de anclaje —el porcentaje de justificaciones cuyas afirmaciones referencian campos que existen realmente en la alerta—, y una manual, sobre una muestra, en la que un revisor humano juzga si el contenido de la explicación es correcto y no solo si cita los campos adecuados. Esta distinción resultó decisiva: es precisamente la que permitió detectar que un anclaje perfecto no garantiza una justificación semánticamente correcta, y la que motivó incorporar la recuperación aumentada descrita en la sección anterior.

Para aislar el efecto de esa incorporación, la campaña se ejecutó dos veces sobre el mismo subconjunto de dieciocho alertas soportadas de la partición de evaluación, con el modelo generativo y la temperatura de muestreo fijadas en ambos casos: una vez con el justificador simple, y otra con la recuperación aumentada activada, comparando en cada caso el mismo par de alertas para poder atribuir cualquier diferencia observada a la sola presencia del módulo de recuperación.

CAPÍTULO 4

RESULTADOS Y DISCUSIÓN

Este capítulo presenta los resultados de la campaña de evaluación de la Fase 6, ejecutada el 2 de septiembre de 2026 sobre la partición de evaluación del conjunto de alertas etiquetado —205 alertas, de las cuales 18 pertenecen al caso de uso acotado y tienen, por tanto, una clase verdadera contra la que se puede medir—, con el perfil de cliente empresarial y el justificador de mil millones de parámetros a temperatura de muestreo nula.

4.1. Resultado principal: clasificación frente al método de referencia

La pregunta que la fase debía responder es si el triaje asistido aporta algo sobre el nivel de regla de Wazuh tomado como método tradicional de referencia. La tabla 4.1 resume el resultado.

Tabla 4.1. Prototipo frente al baseline de Wazuh (partición de evaluación, $n = 205$)

Métrica	Prototipo	Baseline (nivel ≥ 5)
Precisión	0,556	0,154
Exhaustividad	1,000	1,000
Medida F_1	0,714	0,267
Tasa de falsos positivos	0,041	0,282

La matriz de confusión del prototipo es $VP = 10$, $FP = 8$, $VN = 187$, $FN = 0$; la del baseline en su umbral óptimo es $VP = 10$, $FP = 55$, $VN = 140$, $FN = 0$. Ambos métodos detectan la totalidad de las amenazas reales de la partición —exhaustividad de 1,0 en los dos casos—, pero el prototipo reduce la tasa de falsos positivos de 0,282 a 0,041, casi siete veces menos. La diferencia son 187 eventos de ruido de plataforma que el motor clasifica correctamente como no soportados y que el nivel de regla, por sí solo, no tiene forma de suprimir. Esta reducción del ruido operativo sin pérdida de exhaustividad es, precisamente, el resultado que el proyecto se propuso demostrar.

4.2. El límite de precisión del prototipo, y por qué

La precisión del prototipo se queda en 0,556, no en 1,0, por una razón concreta y verificable: las dieciocho alertas soportadas de la partición corresponden todas al mismo activo y al mismo servicio —acceso por SSH al servidor con vulnerabilidades documentadas del laboratorio—, de las cuales diez son intentos de acceso reales y ocho son actividad de un administrador legítimo que se equivocó de credencial. El clasificador determinista decide por familia de alerta y por postura de exposición del activo, y como el servicio está efectivamente expuesto en ese nodo, marca las dieciocho como amenaza potencial: no dispone de ninguna señal sobre la legitimidad del origen que le permita distinguir al administrador del atacante. Es el límite esperado de un clasificador que no incorpora aprendizaje sobre patrones de comportamiento, y es exactamente la limitación que motivaría un clasificador con ajuste fino si el conjunto de datos tuviera variedad suficiente de familias para entrenarlo sin invalidar la propia evaluación.

4.3. Por qué el óptimo del baseline es débil

El nivel de regla de Wazuh se evaluó como clasificador barriendo su umbral de decisión; la tabla 4.2 resume la medida F_1 resultante en cada punto.

Tabla 4.2. Medida F_1 del baseline según el umbral de nivel de regla

nivel $\geq$	3	4	5	6	7	8	9	10	11	12
F_1	0,093	0,267	0,267	0,035	0,035	0,167	0,167	0,167	—	—

El mejor punto del baseline — $F_1 = 0,267$, en los umbrales 4 y 5— no se acerca en ningún caso al resultado del prototipo ($F_1 = 0,714$). La curva, además, no es monótona: subir el umbral no mejora de forma limpia, porque los verdaderos positivos y buena parte del ruido conviven en los mismos niveles de severidad. Esto confirma que el nivel de regla, por sí solo, no basta como criterio de prioridad, y justifica la reclasificación que realiza el motor de triaje.

4.4. Operación y continuidad del servicio

El tiempo de clasificar y decidir sobre una alerta, con el clasificador determinista, fue del orden de 0,01 ms por alerta, muy por debajo del presupuesto de latencia establecido. Justificar con el modelo de lenguaje de mil millones de parámetros tomó, en esta campaña, aproximadamente 36 s por alerta —más que en una invocación aislada, porque cada justifi-

cación recarga el modelo de 808 MB en un proceso nuevo; en un despliegue real el modelo permanecería cargado entre invocaciones—. La cobertura del clasificador —la proporción de alertas que caen dentro del caso de uso acotado— fue de 0,088: solo el 8,8 % de las alertas de un Wazuh recién desplegado corresponden a un patrón clasificable, y el resto es telemetría interna de la propia plataforma. La tasa de escalado a validación humana fue de 0,0 sobre este conjunto, porque el clasificador determinista nunca reportó una confianza por debajo del umbral de escalado en estas alertas.

Sobre la métrica de continuidad —el requisito que distingue a este trabajo de un sistema de respuesta automatizada convencional—, no se registró ninguna acción disruptiva indebida: ninguna acción propuesta a partir de una alerta que resultó ser un falso positivo alcanzaba un servicio que el cliente prestara. El escenario más exigente de esta métrica —una acción de alto impacto que deba ser efectivamente retenida por la validación humana— no llegó a ejercitarse sobre este conjunto de datos, lo que se documenta como una limitación de la muestra y no como una garantía general.

4.5. Calidad de la justificación: anclada, pero inicialmente poco fiable

La verificación automática de anclaje —que ninguna dirección IP mencionada sea distinta de la de la alerta y que el texto referencie al menos un campo concreto de ella— dio una tasa de 18 sobre 18: el cien por ciento de las justificaciones del modelo de mil millones de parámetros pasó la verificación. Sin embargo, la revisión manual de esa misma muestra reveló que un anclaje perfecto no garantiza una explicación correcta. Un ejemplo textual generado por el modelo antes de incorporar recuperación aumentada:

La regla 5760 es un protocolo de seguridad para la autenticación de usuarios en redes. El MITRE T1110.001 es un conjunto de técnicas de seguridad para la autenticación de usuarios...¿

La regla 5760 de Wazuh no es un protocolo: es la regla que detecta un fallo de autenticación por SSH. La técnica T1110.001 no es un conjunto de técnicas de seguridad: es *Password Guessing*, una técnica de ataque del catálogo MITRE ATT&CK. El modelo cita los campos correctos —de ahí que ancle— pero describe su significado al revés, y de las dieciocho justificaciones generadas solo cuatro textos resultaron distintos entre sí, porque una temperatura de muestreo nula sobre un único nodo produce salidas casi idénticas. La conclusión de esta medición es metodológicamente importante: la tasa de anclaje mide que el modelo no inventa campos, no que acierte su significado, y no debe leerse como un indicador de calidad de la justificación.

4.6. El contraste con recuperación aumentada

Sobre las mismas dieciocho alertas, con el mismo modelo y la misma temperatura, se repitió la campaña con el módulo de recuperación aumentada activado. La tasa de anclaje automática no cambió —se mantuvo en 18 sobre 18, porque mide otra cosa—, pero la corrección semántica de la justificación, evaluada manualmente, sí lo hizo. La misma alerta que antes producía el texto citado en la sección anterior, generó con recuperación aumentada:

La regla 5760 de Wazuh indica un ataque de fuerza bruta SSH en curso desde una IP.

correcta y respaldada por los pasajes recuperados del corpus curado sobre las reglas de fuerza bruta SSH. En una alerta de falso positivo, el contraste fue igual de marcado: de la regla 5710 define el protocolo SSH como un protocolo seguro —una afirmación sin sentido— a la regla 5710 de Wazuh indica que se probó un nombre de usuario que no existe por SSH, un indicio de ataque de fuerza bruta, que es la lectura correcta del evento.

Tres matices, medidos y no solo observados, acompañan esta mejora. Primero, la diversidad de textos no aumentó —siguieron saliendo solo cuatro textos distintos de dieciocho—, porque la causa de esa baja diversidad es el propio conjunto de datos concentrado en un nodo, no el mecanismo de justificación. Segundo, la recuperación resultó imprecisa en el identificador exacto de la regla: para una alerta de la regla 5760, el sistema recuperó el grupo completo de reglas de fuerza bruta SSH (5710, 5712, 5763) en lugar del identificador preciso; aun así, el dominio recuperado era el correcto, y por eso la justificación mejoró. Tercero, con recuperación aumentada el modelo tiende a parafrasear los pasajes recuperados —enumerar las reglas del corpus— más que a razonar de forma original sobre la alerta concreta, lo que es preferible a inventar una explicación pero sigue estando lejos de la redacción de un analista humano.

4.7. Verificación sobre el conjunto completo

Para descartar que el resultado dependiera de la partición elegida, la clasificación —sin el justificador, por su coste de tiempo— se repitió sobre las 410 alertas del conjunto completo (36 soportadas). Las tasas obtenidas —precisión 0,556, exhaustividad 1,0, F_1 0,714, tasa de falsos positivos 0,041— resultaron idénticas a las de la partición de evaluación, lo que era el resultado esperado: el clasificador es determinista y no se entrena, así que no existe diferencia entre entrenar y evaluar sobre él. La partición se respetó de todas formas por rigor metodológico.

4.8. Limitaciones y análisis de errores

Cuatro limitaciones, todas medidas y no solo declaradas, acotan el alcance de estos resultados. Primero, el tamaño de la muestra soportada —dieciocho alertas, de una sola familia de ataque sobre un único activo— hace que los porcentajes de precisión se sostengan sobre pocos casos, y que la correlación de orden de prioridad no pudiera calcularse por falta de variedad. Segundo, el error sistemático dominante del prototipo —los ocho falsos positivos— tiene una causa identificada y no un origen difuso: la ausencia de una señal sobre la legitimidad del origen de la conexión. Tercero, el clasificador con ajuste fino permanece bloqueado por la misma concentración del conjunto de datos, así que el desajuste esperado entre un modelo preentrenado sobre prosa y una entrada de eventos estructurados queda documentado, pero no cuantificado. Cuarto, el auditor de vulnerabilidades no es infalible: la ausencia de un hallazgo no demuestra que un activo sea seguro, y la postura de exposición usada como contexto de decisión se tomó de los hallazgos reales de una única campaña de escaneo.

El resultado global, no obstante, es positivo y medido sobre el problema que el proyecto se propuso atacar: el triaje asistido reduce el ruido operativo de forma sustancial sin sacrificar exhaustividad y sin comprometer la continuidad del servicio, y su límite de precisión y su justificador tienen, ambos, una causa identificada y un camino de mejora conocido —un clasificador entrenado con datos de mayor variedad y un modelo de justificación de mayor capacidad—, que la propia interfaz de análisis del prototipo ya permite incorporar sin rediseñar el sistema.

Conclusiones y recomendaciones

El objetivo general de este trabajo era desarrollar un prototipo que clasificara y priorizara alertas de seguridad, justificando cada decisión con un modelo de lenguaje apoyado en generación aumentada por recuperación ejecutada en local, y que redujera los falsos positivos sin sacrificar la detección de amenazas reales. Los resultados del capítulo 4 sostienen que ese objetivo se cumplió dentro del caso de uso acotado sobre el que se evaluó: el prototipo redujo la tasa de falsos positivos frente al nivel de regla de Wazuh en un factor cercano a siete, sin dejar de detectar ninguna amenaza real y sin ejecutar ninguna acción que hubiera interrumpido indebidamente un servicio del cliente. Este resultado confirma, con datos y no solo con diseño, la idea central del trabajo: separar la clasificación, la decisión y la justificación en componentes distintos —una política determinista que decide, un perfil de cliente que filtra, y un modelo de lenguaje que solo explica— permite obtener las ventajas de un modelo de lenguaje sin heredar su principal debilidad, que es decidir de forma no auditable.

La incorporación de recuperación aumentada, no prevista en el diseño original y añadida al detectar en la propia evaluación que el justificador de menor tamaño describía correctamente los campos de una alerta pero se equivocaba en su significado, es la conclusión metodológica más relevante del trabajo: un anclaje verificable perfecto no implica una justificación correcta, y ambas propiedades deben medirse por separado. Corregir el conocimiento del modelo mediante un corpus curado y recuperación por similitud, en vez de sustituirlo por un modelo mayor que el hardware disponible no podía sostener con una latencia razonable, resolvió esa brecha sin añadir ningún requisito de cómputo significativo.

El trabajo deja, con la misma honestidad con que reporta sus logros, dos límites identificados y no resueltos. El primero es la precisión del clasificador determinista, limitada por su incapacidad de distinguir un administrador legítimo de un atacante sobre el mismo servicio: es exactamente el problema que un clasificador con ajuste fino resolvería, y que permanece bloqueado porque el conjunto de datos disponible, concentrado en una sola familia de ataque, no ofrece variedad suficiente para entrenarlo sin invalidar la propia evaluación. El segundo es el alcance estadístico de la evaluación misma: dieciocho alertas soportadas, todas del mismo activo y servicio, sostienen un resultado direccionalmente claro pero no permiten generalizar

sus cifras exactas a un cliente con un perfil de amenazas distinto.

Recomendaciones

- **Ampliar el conjunto de datos etiquetado con más familias de ataque y más activos**, antes de intentar de nuevo el ajuste fino de un clasificador. Es la condición que hoy bloquea la pieza más importante que falta en el prototipo, y no depende de ninguna decisión externa a este proyecto.
- **Evaluar el prototipo con un modelo de justificación de mayor capacidad** en cuanto se disponga de una unidad de procesamiento gráfico o de memoria adicional, aprovechando que la interfaz de análisis ya aísla al motor de triaje del modelo concreto que la implementa: el cambio no exigiría modificar la lógica de decisión.
- **Resolver, junto con Domotai C.A., si el sistema de logs propietario que originalmente motivó el proyecto llegará a estar disponible**, y en qué formato entrega sus eventos. De confirmarse, se integraría como una segunda fuente a través del mismo módulo de ingesta que hoy consume las alertas de Wazuh, sin desplazarlo.
- **Completar la validación sobre un equipo de borde con firmware real**, hoy sustituido en el laboratorio por un contenedor Linux provisional: las acciones del catálogo están diseñadas para ese firmware, pero solo se han verificado sobre su sustituto.
- **Someter la calidad de la justificación a una segunda revisión manual independiente** antes de dar por definitiva la lectura de utilidad reportada en este informe, dado que el método de evaluación de esa dimensión no es completamente automatizable.

Trabajo futuro

Quedan fuera del alcance de esta pasantía, y se documentan explícitamente para que no se interpreten como un olvido: la respuesta automatizada sin validación humana previa, que el diseño actual descarta de forma deliberada; la integración multicapa con fuentes de identidad, correo y nube, que ampliaría la correlación más allá de la red del cliente; el aprendizaje continuo a partir de la retroalimentación que el analista ya registra en cada validación, hoy almacenada pero no utilizada para ajustar ningún modelo; y el despliegue y la escalabilidad del prototipo sobre múltiples clientes de forma simultánea. Cada una de estas líneas es una extensión natural sobre la arquitectura ya construida, y ninguna exige rediseñarla desde cero.

Referencias

- [1] Ehsan Aghaei, Xi Niu, Waseem Shadid, and Ehab Al-Shaer. SecureBERT: A domain-specific language model for cybersecurity. *arXiv preprint*, 2022. [arXiv:2204.02685](#).
- [2] Cisco Foundation AI. Foundation-Sec-8B. <https://huggingface.co/fdtn-ai/Foundation-Sec-8B>, 2025. Ficha del modelo; consultada el 2 de septiembre de 2026.
- [3] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020. [arXiv:2005.11401](#).
- [4] OWASP Foundation. OWASP top 10 for LLM applications. <https://genai.owasp.org/llm-top-10/>, 2025. Consultado el 25 de agosto de 2026.
- [5] Containerlab. Containerlab — documentación oficial. <https://containerlab.dev/>, 2026. Consultado el 23 de agosto de 2026.
- [6] MITRE Engenuity. MITRE ATT&CK Evaluations — enterprise. <https://attackevals.mitre-engenuity.org/>, 2025. Resultados públicos de detección por plataforma; consultado el 25 de agosto de 2026.
- [7] MITRE Corporation. MITRE ATT&CK. <https://attack.mitre.org/>, 2026. Consultado el 25 de agosto de 2026.
- [8] MITRE Corporation. MITRE ATLAS. <https://atlas.mitre.org/>, 2026. Panorama de amenazas contra sistemas de inteligencia artificial; consultado el 25 de agosto de 2026.
- [9] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *arXiv preprint*, 2023. [arXiv:2307.03172](#).
- [10] Meta AI. CyberSecEval: A secure coding benchmark for language models. *arXiv preprint*, 2023. [arXiv:2312.04724](#) — publicado como parte de la iniciativa Purple Llama.